

DRAFT - VERIFY BEFORE APPLYING

Kajal Raju Kale

BCA student | Aspiring software developer

Contact details, college, graduation dates and public profile await Kajal's confirmation.

PROFILE

BCA student preparing for an entry-level software development role. Studying practical frontend, backend, database and data workflows through runnable learning projects.

EDUCATION

Bachelor of Computer Applications (BCA)

College and attendance dates: to be confirmed by Kajal.

LEARNING PORTFOLIO - PROJECT INDEX

01 Animated Portfolio	HTML / CSS / JavaScript
02 Django Task Board	Django / HTMX / SQLite
03 Notebook Chat	FastAPI / SQLite / lexical RAG / optional Ollama
04 Job Application Tracker	Streamlit / SQLite
05 Study Planner	Streamlit / SQLite
06 Data Cleaning Studio	Streamlit / Pandas / public APIs

PROJECT OWNERSHIP AND NEXT STEPS

These projects are prepared learning examples. Kajal should run, inspect and modify them, then identify only her own contributions and skills she can explain. Add verified contact information, education dates and a personal GitHub profile before applying.

Animated Portfolio

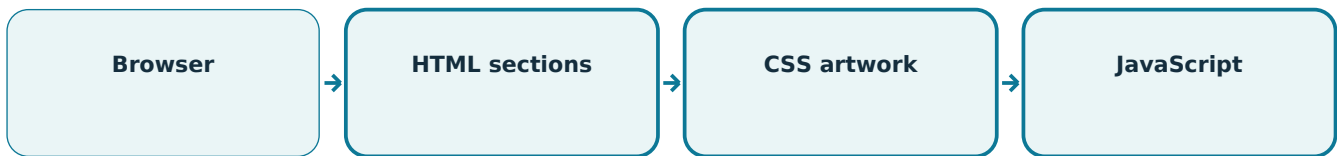
HTML / CSS / JavaScript

A responsive concept site for a fictional studio. Semantic sections, CSS artwork and lightweight JavaScript create a navigable portfolio without a build step.

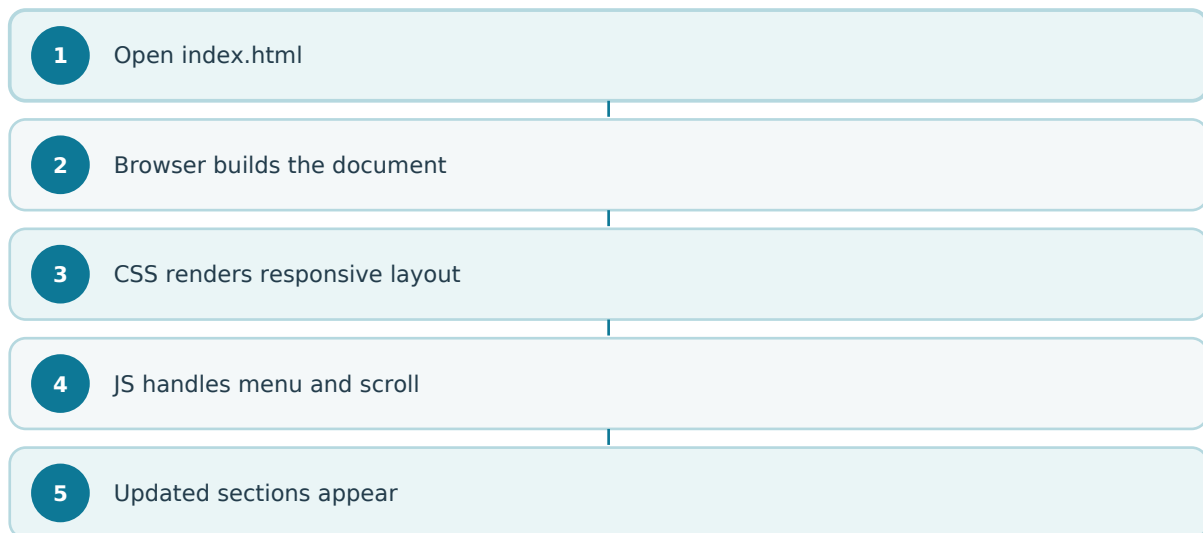
PROJECT STRUCTURE

<code>index.html</code>	Semantic sections and navigation
<code>styles.css</code>	Layout, artwork and animations
<code>script.js</code>	Menu and scroll reveal behavior

ARCHITECTURE IMAGE



DATA FLOW IMAGE



IMPLEMENTATION

The browser loads three local files directly. HTML defines landmarks and section anchors; CSS handles both the artwork and responsive breakpoints. JavaScript changes navigation state on small screens and reveals sections as they enter the viewport. A reduced-motion media query removes unnecessary movement for users who request it.

Example: A visitor selects Work in the menu; the page moves to the case studies and the relevant content becomes visible.

KEY FEATURES

Responsive navigation, accessible keyboard flow, reveal animations and reduced-motion styling.

SCOPE AND LIMITS

Fictional content and demonstration contact address; replace these before public use.

RUN LOCALLY

Open `index.html` locally

[Source code and full setup guide](#)

Django Task Board

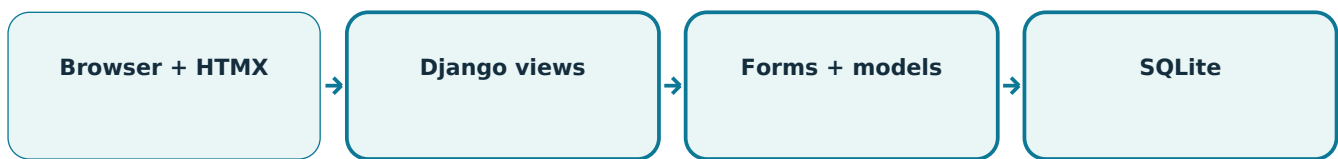
Django / HTMX / SQLite

A local task manager with server-rendered updates. Django owns validation and persistence; HTMX swaps a task-list fragment after actions.

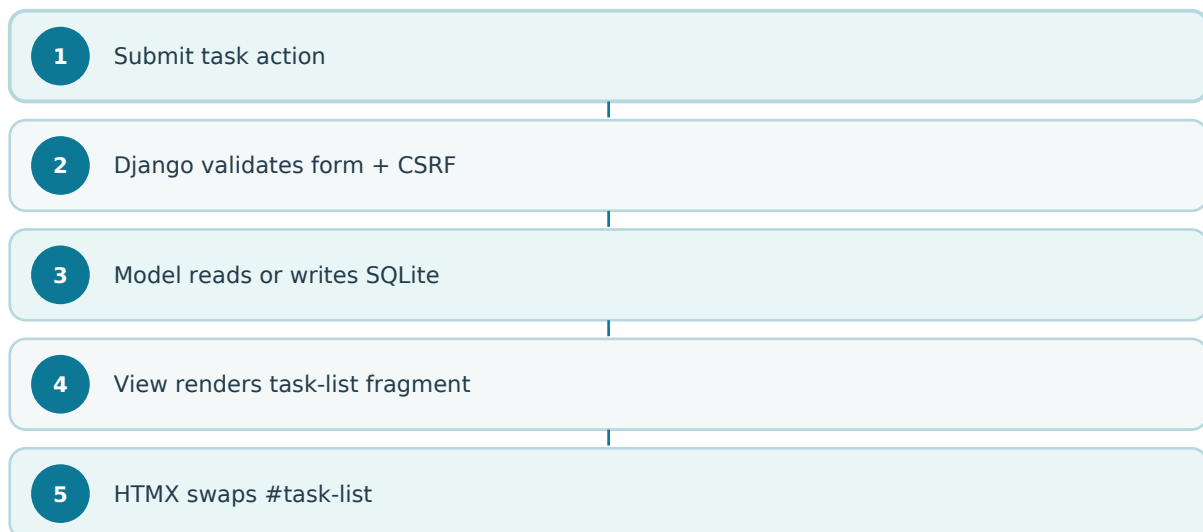
PROJECT STRUCTURE

board/models.py	Task model and database fields
board/forms.py	Input validation
board/views.py	Requests and partial responses
board/templates/	Page and task-list fragments

ARCHITECTURE IMAGE



DATA FLOW IMAGE



IMPLEMENTATION

The page submits actions to Django views. A Django form validates incoming fields before the model writes tasks to SQLite. For HTMX requests, the view renders only the task-list template; the browser replaces that fragment while leaving the rest of the page in place. Mutating requests include Django's CSRF token.

Example: When a task is marked complete, Django updates its row and returns the refreshed list fragment without a full page reload.

KEY FEATURES

Create, filter, complete and delete tasks; local HTMX asset; CSRF checks and escaped templates.

SCOPE AND LIMITS

Single-user development example. Needs authentication and production settings before deployment.

RUN LOCALLY

`python manage.py migrate; python manage.py runserver 127.0.0.1:8000`

[Source code and full setup guide](#)

Notebook Chat

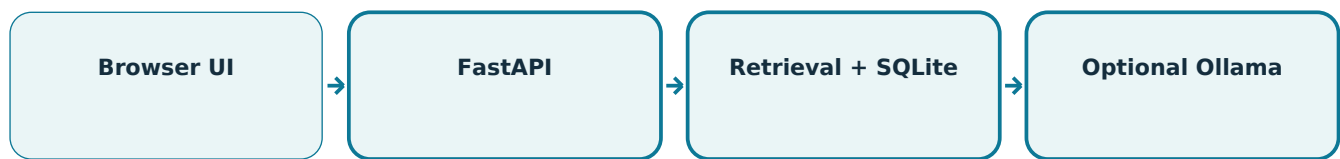
FastAPI / SQLite / lexical RAG / optional Ollama

A local knowledge-base chat that retrieves passages from Markdown or text notes and shows citations. Optional Ollama generation uses retrieved context.

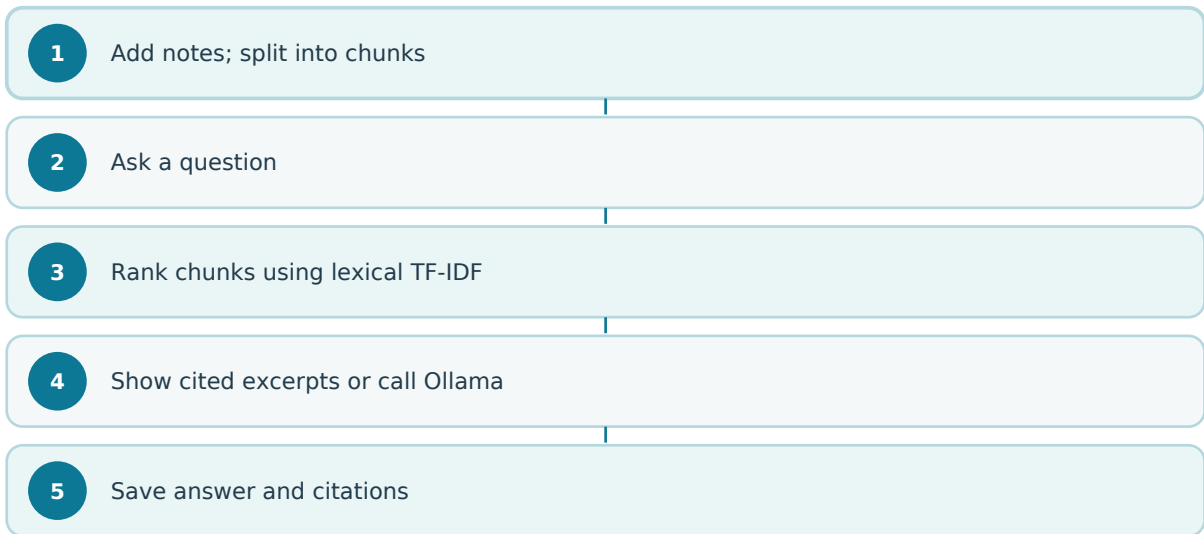
PROJECT STRUCTURE

<code>index.html + client.js</code>	Chat and knowledge library UI
<code>app.py</code>	API, chat and persistence
<code>retrieval.py</code>	Chunking and TF-IDF ranking
<code>knowledge/</code>	Bundled example notes

ARCHITECTURE IMAGE



DATA FLOW IMAGE



IMPLEMENTATION

The backend splits notes into passages and scores them with lexical TF-IDF. A question selects the strongest passages, which become cited excerpts in the fallback response. When configured, the backend sends those passages to a local Ollama server for an optional answer. SQLite retains documents, conversations and citation records.

Example: Ask a question about a stored note; the response points back to the passages used, so the reader can inspect the evidence.

KEY FEATURES

Local document ingestion, passage ranking, saved conversations and citations; useful without a model.

SCOPE AND LIMITS

Lexical search may miss synonyms. Model output needs verification; no login or multi-user isolation.

RUN LOCALLY

`uvicorn app:app --host 127.0.0.1 --port 8001`

[Source code and full setup guide](#)

Job Application Tracker

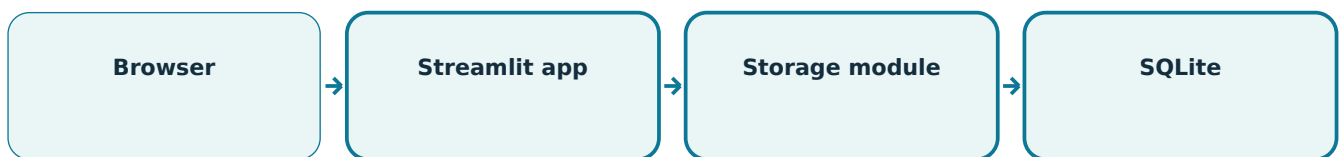
Streamlit / SQLite

A local dashboard for managing applications and interview notes. Streamlit renders the interface and SQLite retains records between sessions.

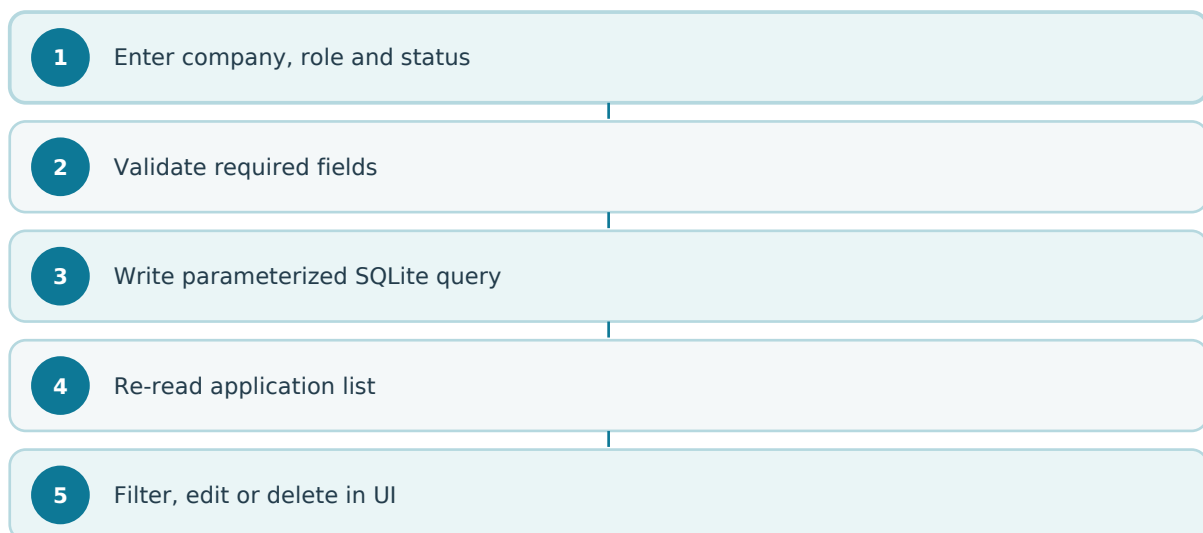
PROJECT STRUCTURE

<code>app.py</code>	Forms, filters, tabs and metrics
<code>application_store.py</code>	Validation and SQLite queries
<code>tests/test_storage.py</code>	Temporary-database lifecycle check

ARCHITECTURE IMAGE



DATA FLOW IMAGE



IMPLEMENTATION

Streamlit forms gather a company, role, status and optional notes. The storage module trims and validates input, then uses parameterized SQL to insert or update a row. On rerun, the app reads the database again and computes metrics and filtered lists from those records. Each app has its own database file next to its source.

Example: After an interview, change an application status to Interview and see the dashboard count update.

KEY FEATURES

Add, update, delete, search and status-filter records; metrics for interviews and offers.

SCOPE AND LIMITS

Local single-user learning app. Database file should be backed up and kept out of Git.

RUN LOCALLY

```
streamlit run app.py --server.port 8501
```

[Source code and full setup guide](#)

Study Planner

Streamlit / SQLite

A local coursework planner with status counts. A small storage module encapsulates validation and persistent task operations.

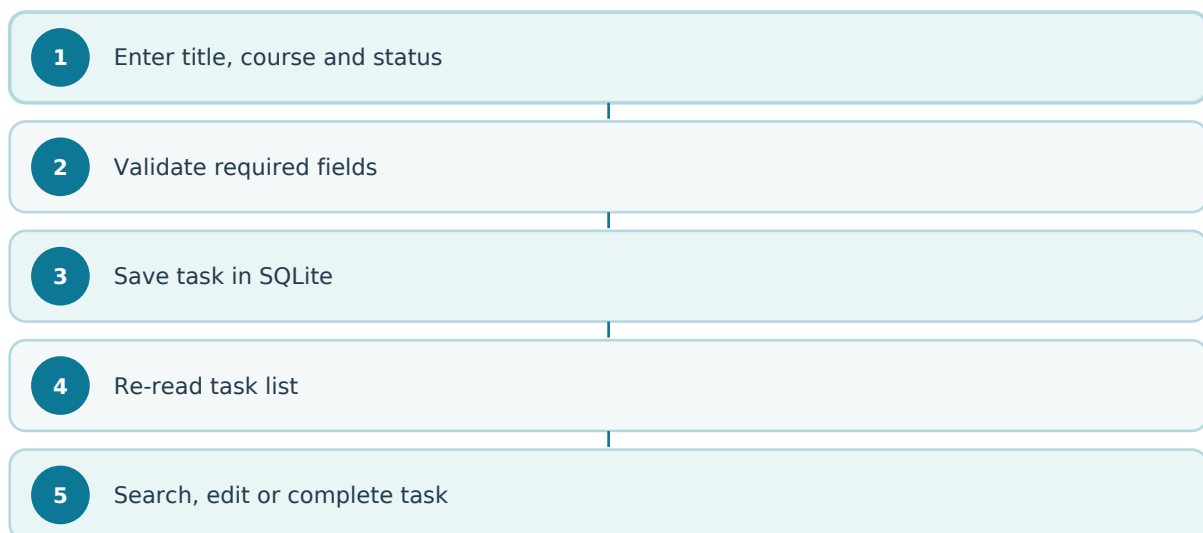
PROJECT STRUCTURE

<code>app.py</code>	Task forms, filters and metrics
<code>planner_store.py</code>	Task validation and SQLite
<code>tests/test_storage.py</code>	Create, edit and delete checks

ARCHITECTURE IMAGE



DATA FLOW IMAGE



IMPLEMENTATION

The UI separates task review, creation and editing into tabs. The storage module requires a title, course and supported status, and persists data using parameterized SQLite statements. Search checks task titles, course names and notes; progress metrics are calculated from the current database rows after each interaction.

Example: Add a coursework task, then change it from To do to Done and verify the progress count.

KEY FEATURES

Create and edit coursework, track progress, search notes and filter by task status.

SCOPE AND LIMITS

Single-user local demo, without accounts or shared access.

RUN LOCALLY

```
streamlit run app.py --server.port 8502
```

[Source code and full setup guide](#)

Data Cleaning Studio

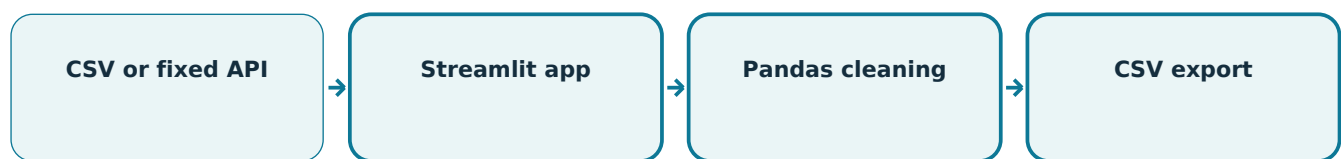
Streamlit / Pandas / public APIs

An interactive data workflow with a fictional offline sample, CSV upload and fixed public sources for weather, currency rates and economic indicators.

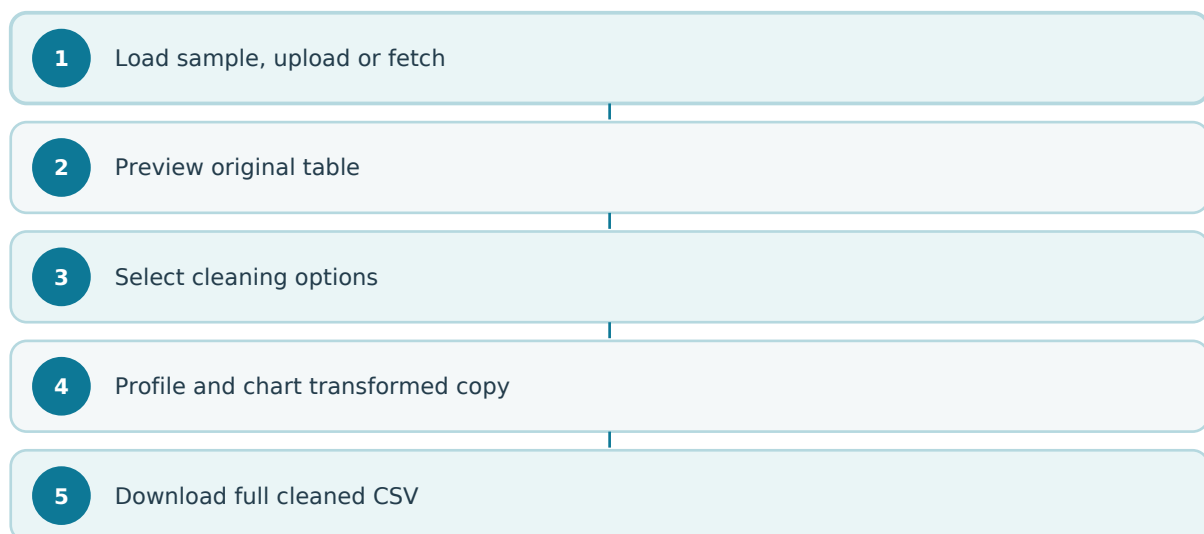
PROJECT STRUCTURE

app.py	Source selection and cleaning controls
sources.py	Open-Meteo, ECB, World Bank clients
cleaning.py	Pure Pandas transformations
samples/	Offline demonstration dataset

ARCHITECTURE IMAGE



DATA FLOW IMAGE



IMPLEMENTATION

A source adapter turns each supported API response or CSV input into a Pandas table. A pure cleaning function copies the original table before applying selected transformations, so previewing alternatives does not alter the source. Column profiles expose types, missing values and unique counts; the final table is serialized to CSV for download.

Example: Load the fictional sales sample, convert revenue to numbers, fill numeric medians and compare missing-cell counts before exporting.

KEY FEATURES

Trim text, deduplicate, parse dates and numbers, handle missing values and inspect before/after metrics.

SCOPE AND LIMITS

Public APIs need internet access; previews show 500 rows while the download retains all rows.

RUN LOCALLY

```
streamlit run app.py --server.port 8503
```

[Source code and full setup guide](#)